

Protokol: Algoritmické přístupy k řešení zobecněného Sudoku

Marek Sobotka

8.5.2026

1 Úvod

Cílem tohoto projektu je modelovat, generovat a řešit zobecněné mřížky Sudoku o rozměrech $N \times N$ (kde $N \in \{9, 16, 25, 36, 49\}$). Zatímco standardní Sudoku 9x9 představuje pro moderní počítače triviální úlohu, exponenciální nárůst stavového prostoru při zvětšování mřížky transformuje tento hlavolam na rigorózní testovací prostředí pro NP-úplnou kombinatorickou optimalizaci.

Tento protokol dokumentuje formulaci Sudoku jako problému splňování omezujících podmínek (CSP), počáteční pokusy o jeho řešení pomocí celočíselného lineárního programování (ILP), následný výkonnostní bottleneck při škálování a konečnou, vysoce optimalizovanou implementaci využívající čistou výrokovou splnitelnost (SAT) skrze konjunktivní normální formu (CNF).

2 Teoretický základ a formulace

2.1 Sudoku jako CSP (Constraint Satisfaction Problem)

V samotném jádru je Sudoku čistým problémem splňování omezujících podmínek.

- **Proměnné:** Prázdné buňky v mřížce $N \times N$.
- **Doména:** Celá čísla $\{1, 2, \dots, N\}$.
- **Omezení:** Podmínka `AllDifferent` aplikovaná napříč každým řádkem, každým sloupcem a každou pod-mřížkou o velikosti $\sqrt{N} \times \sqrt{N}$.

2.2 Formulace celočíselného lineárního programování (ILP)

Pro matematické řešení byl CSP nejprve převeden na problém přesného pokrytí (Exact Cover) pomocí ILP. Tato formulace využívá 3D tenzor binárních proměnných $x_{r,c,v} \in \{0, 1\}$, kde proměnná nabývá hodnoty 1, pokud je hodnota v umístěna v řádku r a sloupci c . Omezení byla definována pomocí lineárních rovnic:

Omezení buňky (právě jedna hodnota v každé buňce):

$$\sum_{v=1}^N x_{r,c,v} = 1 \quad \forall r, c$$

Obdobná sumační omezení byla aplikována na řádky, sloupce a bloky, aby bylo zaručeno, že se každá hodnota vyskytne v dané oblasti právě jednou.

2.3 Formulace výrokové splnitelnosti (SAT)

Vzhledem k tomu, že ILP spoléhá na operace s floating point čísly a spojitou relaxaci, byl problém přeformulován do čisté výrokové logiky. Podmínky byly zakódovány v konjunktivní normální formě (CNF).

S využitím stejného konceptu 3D tenzoru je binární proměnná $s_{x,y,z}$ rovna **True**, pokud je hodnota z v buňce (x, y) . Pravidla byla přeložena do logických hradel. Například pravidlo „maximálně jeden výskyt hodnoty z v řádku x “ je vyjádřeno jako párové negativní klauzule:

$$\bigwedge_{y_1=1}^{N-1} \bigwedge_{y_2=y_1+1}^N (\neg s_{x,y_1,z} \vee \neg s_{x,y_2,z})$$

Významná část informací pochází z tohoto článku.

3 Experimenty a iterativní optimalizace

Řešiče byly implementovány v jazyce Julia. Jak se velikost hrací plochy zvětšovala z 9x9 na 25x25 a více, narazili jsme na několik fundamentálních výkonnostních bariér, které si vyžádaly algoritmické úpravy.

3.1 Experiment 1: ILP

Původní řešič využíval modelovací jazyk JuMP.jl ve spojení s open-source řešičem HiGHS.

- **Pozorování:** Pro $N = 9$ a $N = 16$ fungoval ILP řešič adekvátně a vyřešil mřížky ve zlomcích sekundy. Po přechodu na $N = 25$ (což vyžaduje 15 625 proměnných) však doba běhu dramaticky vzrostla na přibližně 100 sekund.
- **Analýza:** ILP engine plýtvá obrovským množstvím výpočetních zdrojů aplikací simplexové metody a prohledáváním stavového prostoru metodou branch-and-bound na problém, který neobsahuje žádnou účelovou (optimalizační) funkci. Režie matematiky s floating point aritmetikou se ukázala být naprosto nevhodná pro rozsáhlé čisté logické hlavolamy.

3.2 Experiment 2: Přejít na SAT a paměťová režie

S cílem vyhnout se matematické režii ILP byla formulace převedena na výrokovou splnitelnost (SAT). Zpočátku byl ke generování CNF klauzulí použit vysokoúrovňový wrapper Satisfiability.jl napojený na solver Z3.

- **Pozorování:** Ačkoliv byla tato formulace matematicky nadřazená, implementace přes wrapper byla neuvěřitelně pomalá. Mřížka 25x25 vyžadovala vygenerování více než 750 000 jednotlivých logických objektů v jazyce Julia.
- **Analýza:** Bottleneck se přesunul ze samotného řešiče na Garbage Collector jazyka. Přidělování paměti a sestavování abstraktních syntaktických stromů (AST) pro stovky tisíc logických klauzulí trvalo nepřijatelně dlouho ještě předtím, než SAT řešič v C++ vůbec zahájil výpočet.

3.3 Experiment 3: Nativní generování DIMACS (Konečné řešení)

Pro dosažení skutečné škálovatelnosti bylo zcela obejito vysokoúrovňové přidělování paměti. Generátor podmínek byl přepsán tak, aby mapoval 3D booleovské proměnné na 1D celočíselné ID a zapisoval CNF klauzule jako surové textové řetězce přímo do souboru ve standardním formátu **DIMACS**. Tento soubor byl následně přesměrován do vysoce optimalizovaného C++ SAT engine (Microsoft Z3 spuštěný v DIMACS módu).

- **Pozorování:** Odstranění režie jazyka vedlo k masivnímu zrychlení. Zápis více než 750 000 klauzulí trval pouhé milisekundy a samotný SAT řešič zpracoval logická hradla téměř okamžitě. Tímto postupem se podařilo srazit čas pro mřížku 25x25 k jedné sekundě a úspěšně škálovat řešení i na velikosti 36x36 a 49x49.

4 Konečné srovnání a výsledky

Poznámka: Všechny testy byly spuštěny na hardwaru AMD Ryzen 5 3600, 16GB RAM v prostředí Julia v1.12.6

Velikost mřížky	Proměnné	ILP (HiGHS)	SAT (DIMACS + Z3)	Zrychlení
9x9	729	0.067 s	0.135 s	– 0.068 s
16x16	4 096	0.35 s	0.19 s	0.16 s
25x25	15 625	77 s	1.5 s	75.5 s
36x36	46 656	DNF (Timeout)	4 s	N/A
49x49	117 649	DNF (Timeout)	15 s	N/A

Tabulka 1: Porovnání výkonu ILP a SAT řešiče na různých velikostech Sudoku.

4.1 Optimalizace generátoru (MRV)

Se zvětšující se hrací plochou se exponenciálnímu zpomalení nevyhnul ani samotný *generátor* hádanek. Použití naivního backtrackingu vedlo u velikosti 49x49 k nekonečným smyčkám. Problém byl vyřešen nasazením heuristiky **Minimum Remaining Values (MRV)**. Generátor nyní vždy vybírá prázdnou buňku s nejmenším počtem legálních možností, což okamžitě prořezává slepé větve prohledávacího stromu a umožňuje generování obřích ploch v rozumném čase.

5 Závěr

Provedené experimenty přesvědčivě ukazují, že ačkoliv je celočíselné lineární programování (ILP) mimořádně silným nástrojem pro spojitou a nákladovou optimalizaci, je z fundamentálního hlediska nevhodné pro husté logické problémy, jakými je rozsáhlé Sudoku.

Přísným překladem omezujících podmínek do výrokové splnitelnosti (SAT) a eliminací bottlenecků jazyka pomocí nativního generování DIMACS souboru se výkon zlepšil o několik řádů.